

RISC-V Core Datasheet (RTL As-Implemented)

2026-07-20

RISC-V Core Datasheet (RTL As-Implemented)

1. Document Info

- Project: RISC-V training core
- RTL baseline: source/sv/rtl
- Datasheet date: 2026-07-20
- ISA target: RV32I subset (M-mode only, no CSR/trap implementation)
- Endianness: little-endian

2. Executive Summary

This core is a simple in-order 32-bit RISC-V pipeline intended for bring-up and functional verification. The implementation is organized in three stages:

- IF: instruction fetch and IF/ID register
- ID/EX: decode, execute, branch/jump resolve, memory request generation
- WB: load/store memory access and register file write-back

Instruction memory is instantiated inside IF stage. Data memory is instantiated inside WB stage.

3. Top-Level Integration

Module: riscv_top

- Inputs: clk_i, rst_ni
- Outputs: illegal_instr_o
- Parameters:
- IMEM_WORDS (default 1024)
- IMEM_HEX_FILE (default empty string)

Module: riscv

- Inputs: clk_i, rst_ni
- Outputs: illegal_instr_o
- Internal submodules:
- riscv_if_stage
- riscv_id_ex_stage
- riscv_wb_stage
- riscv_regfile

4. Microarchitecture

4.1 Pipeline

- 3-stage in-order pipeline
- No branch prediction
- Branch and jump resolution in ID/EX
- IF flush when redirect is taken

4.2 IF Stage

Module: riscv_if_stage

- Owns architectural PC register pc_q
- Fetches from riscv_imem using zero-wait combinational response model
- Redirect logic priority:
 - if pc_redirect_valid_i: pc_q <= pc_redirect_addr_i
 - else: pc_q <= pc_q + 4
- IF/ID register outputs:
 - if_valid_o
 - if_pc_o
 - if_instr_o
- On reset and flush, IF/ID instruction set to NOP (0x00000013)

4.3 ID/EX Stage

Module: riscv_id_ex_stage

- Decodes opcode/funct3/funct7 and immediates (I/B/J/U/S)
- Computes ALU and control-flow results
- Generates registered outputs toward WB
- Generates illegal_instr_o for unsupported encodings
- Memory ops are decoded here and turned into mem_* + load_* control payload

4.4 WB Stage

Module: riscv_wb_stage

- Instantiates riscv_dmem
- Muxes ALU result vs load result
- Blocks writes to x0
- Registers final register-file write interface:
 - rf_we_o
 - rf_waddr_o
 - rf_wdata_o

4.5 Register File

Module: riscv_regfile

- 32 x 32-bit integer registers
- 2 asynchronous read ports
- 1 synchronous write port

- x0 hardwired to zero

5. Functional Units

- riscv_alu_addsub: add/sub with operand isolation
- riscv_alu_shift: sll/srl/sra with operand isolation
- riscv_alu_logic: xor/or/and with operand isolation
- riscv_alu_compare: signed/unsigned less-than comparator

6. Instruction Support (Implemented)

6.1 R-type

- add, sub
- sll, slt, sltu
- xor, srl, sra
- or, and

6.2 I-type ALU

- addi, slti, sltiu
- xori, ori, andi
- slli, srli, srai

6.3 U/J/B-type

- lui, auipc
- jal, jalr
- beq, bne, blt, bge, bltu, bgeu

6.4 Load/Store

- Loads: lb, lh, lw, lbu, lhu
- Stores: sb, sh, sw

7. Not Implemented

- ecall, ebreak
- CSR instructions (csrrw/csrrs/csrrc/csrrwi/csrrsi/csrrci)
- fence, fence.i
- exceptions/traps pipeline handling (beyond illegal flag)

8. Memory Subsystem Model

8.1 IMEM

Module: riscv_imem

- Depth: IMEM_WORDS x 32 bits

- Optional preload through \$readmemh(IMEM_HEX_FILE, imem)
- Combinational read
- Out-of-range or misaligned fetch returns NOP

8.2 DMEM

Module: riscv_dmem

- Depth: DMEM_WORDS x 32 bits (default 1024)
- Synchronous writes
- Combinational reads
- Width support:
 - 00 byte
 - 01 halfword
 - 10 word
- Sign/zero extension on load controlled by sign_ext_i
- Misaligned/out-of-range reads currently return zero

9. Hazard and Forwarding Behavior

Current forwarding in riscv:

- WB-to-EX forwarding from wb_rf_* to rs1_data_fwd/rs2_data_fwd
- EX-to-EX forwarding from id_wb_*_q to rs1_data_fwd/rs2_data_fwd
- Priority: EX-to-EX over WB-to-EX

Coverage:

- Resolves common ALU RAW dependencies including back-to-back ALU producer-consumer

Known limitation:

- Load-use hazard is not explicitly stalled
- A dependent instruction immediately after a load can require dedicated stall/bypass logic depending on timing

10. Control-Flow Behavior

- Branches resolved in ID/EX
- Redirect target computed in ID/EX and registered out
- IF flush on taken redirect to clear wrong-path IF/ID content
- jal writes rd = pc + 4
- jalr writes rd = pc + 4 and clears target bit 0

11. Reset Behavior

- Active-low reset rst_ni
- IF stage resets PC and IF/ID outputs to known values
- ID/EX and WB stage outputs reset to zero
- Register file resets all registers to zero and keeps x0 tied to zero
- DMEM initializes all words to zero (initial block)
- IMEM initializes to NOP unless preloaded by hex

12. Verification Status (From Included TB)

Testbench: `source/sv/tb/riscv_top_tb.sv`

- Clock: 10 ns period
- Reset: deassert after 5 cycles
- Run window: 150 cycles after reset release
- Checks:
- `illegal_instr_o` remains low
- register expected values for x1, x2, x3, x4, x5, x6, x7, x8, x9, x10, x11, x12, x13, x14, x15, x20
- `DMEM[0]` and `DMEM[4]` expected content

Program source for test intent: `source/sv/rtl/imem_test_sequence.md`

13. Interface Signals (Pipeline Boundaries)

IF -> ID/EX

- `if_valid_q`
- `if_pc_q`
- `if_instr_q`

ID/EX -> IF (redirect)

- `id_pc_redirect_valid_q`
- `id_pc_redirect_addr_q`

ID/EX -> WB

- `id_wb_rd_q`
- `id_wb_data_q`
- `id_wb_we_q`
- `mem_req_q`
- `mem_addr_q`
- `mem_we_q`
- `mem_width_q`
- `mem_sign_ext_q`
- `mem_wdata_q`
- `load_valid_q`
- `load_rd_q`

WB -> Regfile

- `wb_rf_we`
- `wb_rf_waddr`
- `wb_rf_wdata`

14. Known RTL Notes

- In `riscv_wb_stage` there is no explicit `wb_valid_i` input. Validity is currently gated in top-level regfile write-enable path using `id_wb_valid_q`.
- This coupling works for current bring-up flow but is structurally weaker than having a fully stage-local valid in WB.
- Header comments in `riscv_id_ex_stage` mention some operations as planned, but decode logic shows they are implemented.

15. Recommendations for Next Revision

- Add explicit load-use hazard detection and pipeline stall/kill handling
- Move/standardize valid propagation with a dedicated per-stage valid protocol
- Introduce interface bundles for stage-to-stage payload to improve maintainability
- Add trap/exception handling path (illegal, misaligned, access faults)
- Add regression with directed hazard tests and randomized instruction streams